arXiv:2201.03545v2 [cs.CV] 2 Mar 2022

A ConvNet for the 2020s

Zhuang Liu[1,2*] Hanzi Mao[1] Chao-Yuan Wu[1] Christoph Feichtenhofer[1] Trevor Darrell[2] Saining Xie[1†]

[1]Facebook AI Research (FAIR) [2]UC Berkeley

Code: https://github.com/facebookresearch/ConvNeXt

Abstract

The "Roaring 20s" of visual recognition began with the introduction of Vision Transformers (ViTs), which quickly superseded ConvNets as the state-of-the-art image classification model. A vanilla ViT, on the other hand, faces difficulties when applied to general computer vision tasks such as object detection and semantic segmentation. It is the hierarchical Transformers (e.g., Swin Transformers) that reintroduced several ConvNet priors, making Transformers practically viable as a generic vision backbone and demonstrating remarkable performance on a wide variety of vision tasks. However, the effectiveness of such hybrid approaches is still largely credited to the intrinsic superiority of Transformers, rather than the inherent inductive biases of convolutions. In this work, we reexamine the design spaces and test the limits of what a pure ConvNet can achieve. We gradually "modernize" a standard ResNet toward the design of a vision Transformer, and discover several key components that contribute to the performance difference along the way. The outcome of this exploration is a family of pure ConvNet models dubbed ConvNeXt. Constructed entirely from standard ConvNet modules, ConvNeXts compete favorably with Transformers in terms of accuracy and scalability, achieving 87.8% ImageNet top-1 accuracy and outperforming Swin Transformers on COCO detection and ADE20K segmentation, while maintaining the simplicity and efficiency of standard ConvNets.

1. Introduction

Looking back at the 2010s, the decade was marked by the monumental progress and impact of deep learning. The primary driver was the renaissance of neural networks, particularly convolutional neural networks (ConvNets). Through the decade, the field of visual recognition successfully shifted from engineering features to designing (ConvNet) architectures. Although the invention of back-propagation-trained ConvNets dates all the way back to the 1980s [42], it was not until late 2012 that we saw its true potential for visual feature learning. The introduction of AlexNet [40] precipitated the "ImageNet moment" [59], ushering in a new era of computer vision. The field has since evolved at a rapid speed. Representative ConvNets like VGGNet [64], Inceptions [68], ResNe(X)t [28, 87], DenseNet [36], MobileNet [34], EfficientNet [71] and RegNet [54] focused on different aspects of accuracy, efficiency and scalability, and popularized many useful design principles.

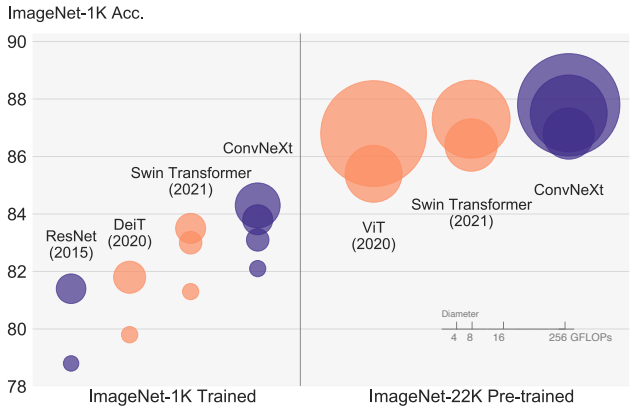


Figure 1. **ImageNet-1K classification** results for ● ConvNets and ○ vision Transformers. Each bubble's area is proportional to FLOPs of a variant in a model family. ImageNet-1K/22K models here take 224^2/384^2 images respectively. ResNet and ViT results were obtained with improved training procedures over the original papers. We demonstrate that a standard ConvNet model can achieve the same level of scalability as hierarchical vision Transformers while being much simpler in design.

The full dominance of ConvNets in computer vision was not a coincidence: in many application scenarios, a "sliding window" strategy is intrinsic to visual processing, particularly when working with high-resolution images. ConvNets have several built-in inductive biases that make them well-suited to a wide variety of computer vision applications. The most important one is translation equivariance, which is a desirable property for tasks like objection detection. ConvNets are also inherently efficient due to the fact that when used in a sliding-window manner, the computations are shared [62]. For many decades, this has been the default use of ConvNets, generally on limited object categories such as digits [43], faces [58, 76] and pedestrians [19, 63]. Entering the 2010s,

*Work done during an internship at Facebook AI Research.
†Corresponding author.

the region-based detectors [23, 24, 27, 57] further elevated ConvNets to the position of being the fundamental building block in a visual recognition system.

Around the same time, the odyssey of neural network design for natural language processing (NLP) took a very different path, as the Transformers replaced recurrent neural networks to become the dominant backbone architecture. Despite the disparity in the task of interest between language and vision domains, the two streams surprisingly converged in the year 2020, as the introduction of Vision Transformers (ViT) completely altered the landscape of network architecture design. Except for the initial "patchify" layer, which splits an image into a sequence of patches, ViT introduces no image-specific inductive bias and makes minimal changes to the original NLP Transformers. One primary focus of ViT is on the scaling behavior: with the help of larger model and dataset sizes, Transformers can outperform standard ResNets by a significant margin. Those results on image classification tasks are inspiring, but computer vision is not limited to image classification. As discussed previously, solutions to numerous computer vision tasks in the past decade depended significantly on a sliding-window, fully-convolutional paradigm. Without the ConvNet inductive biases, a vanilla ViT model faces many challenges in being adopted as a generic vision backbone. The biggest challenge is ViT's global attention design, which has a quadratic complexity with respect to the input size. This might be acceptable for ImageNet classification, but quickly becomes intractable with higher-resolution inputs.

Hierarchical Transformers employ a hybrid approach to bridge this gap. For example, the "sliding window" strategy (*e.g*. attention within local windows) was reintroduced to Transformers, allowing them to behave more similarly to ConvNets. Swin Transformer [45] is a milestone work in this direction, demonstrating for the first time that Transformers can be adopted as a generic vision backbone and achieve state-of-the-art performance across a range of computer vision tasks beyond image classification. Swin Transformer's success and rapid adoption also revealed one thing: the essence of convolution is not becoming irrelevant; rather, it remains much desired and has never faded.

Under this perspective, many of the advancements of Transformers for computer vision have been aimed at bringing back convolutions. These attempts, however, come at a cost: a naive implementation of sliding window self-attention can be expensive [55]; with advanced approaches such as cyclic shifting [45], the speed can be optimized but the system becomes more sophisticated in design. On the other hand, it is almost ironic that a ConvNet already satisfies many of those desired properties, albeit in a straightforward, no-frills way. The only reason ConvNets appear to be losing steam is that (hierarchical) Transformers surpass them in many vision tasks, and the performance difference is usually attributed to the superior scaling behavior of Transformers, with multi-head self-attention being the key component.

Unlike ConvNets, which have progressively improved over the last decade, the adoption of Vision Transformers was a step change. In recent literature, system-level comparisons (*e.g*. a Swin Transformer *vs*. a ResNet) are usually adopted when comparing the two. ConvNets and hierarchical vision Transformers become different and similar at the same time: they are both equipped with similar inductive biases, but differ significantly in the training procedure and macro/micro-level architecture design. In this work, we investigate the architectural distinctions between ConvNets and Transformers and try to identify the confounding variables when comparing the network performance. Our research is intended to bridge the gap between the pre-ViT and post-ViT eras for ConvNets, as well as to test the limits of what a pure ConvNet can achieve.

To do this, we start with a standard ResNet (*e.g*. ResNet-50) trained with an improved procedure. We gradually "modernize" the architecture to the construction of a hierarchical vision Transformer (*e.g*. Swin-T). Our exploration is directed by a key question: *How do design decisions in Transformers impact ConvNets' performance?* We discover several key components that contribute to the performance difference along the way. As a result, we propose a family of *pure ConvNets* dubbed ConvNeXt. We evaluate ConvNeXts on a variety of vision tasks such as ImageNet classification [17], object detection/segmentation on COCO [44], and semantic segmentation on ADE20K [92]. Surprisingly, ConvNeXts, constructed entirely from standard ConvNet modules, compete favorably with Transformers in terms of accuracy, scalability and robustness across all major benchmarks. ConvNeXt maintains the efficiency of standard ConvNets, and the fully-convolutional nature for both training and testing makes it extremely simple to implement.

We hope the new observations and discussions can challenge some common beliefs and encourage people to rethink the importance of convolutions in computer vision.

2. Modernizing a ConvNet: a Roadmap

In this section, we provide a trajectory going from a ResNet to a ConvNet that bears a resemblance to Transformers. We consider two model sizes in terms of FLOPs, one is the ResNet-50 / Swin-T regime with FLOPs around 4.5×10^9 and the other being ResNet-200 / Swin-B regime which has FLOPs around 15.0×10^9. For simplicity, we will present the results with the ResNet-50 / Swin-T complexity models. The conclusions for higher capacity models are consistent and results can be found in Appendix C.

At a high level, our explorations are directed to investigate and follow different levels of designs from a Swin Transformer while maintaining the network's simplicity as a standard ConvNet. The roadmap of our exploration is as

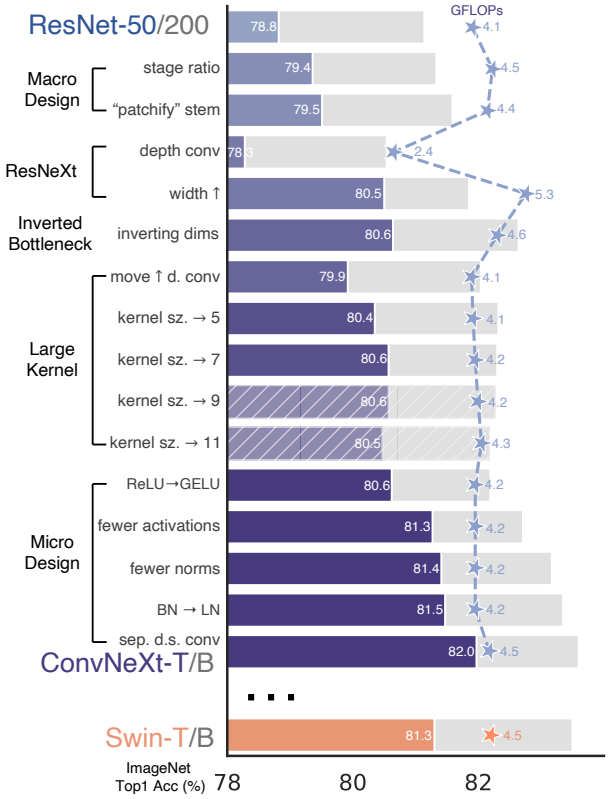


Figure 2. We modernize a standard ConvNet (ResNet) towards the design of a hierarchical vision Transformer (Swin), without introducing any attention-based modules. The foreground bars are model accuracies in the ResNet-50/Swin-T FLOP regime; results for the ResNet-200/Swin-B regime are shown with the gray bars. A hatched bar means the modification is not adopted. Detailed results for both regimes are in the appendix. Many Transformer architectural choices can be incorporated in a ConvNet, and they lead to increasingly better performance. In the end, our pure ConvNet model, named ConvNeXt, can outperform the Swin Transformer.

follows. Our starting point is a ResNet-50 model. We first train it with similar training techniques used to train vision Transformers and obtain much improved results compared to the original ResNet-50. This will be our baseline. We then study a series of design decisions which we summarized as 1) macro design, 2) ResNeXt, 3) inverted bottleneck, 4) large kernel size, and 5) various layer-wise micro designs. In Figure 2, we show the procedure and the results we are able to achieve with each step of the "network modernization". Since network complexity is closely correlated with the final performance, the FLOPs are roughly controlled over the course of the exploration, though at intermediate steps the FLOPs might be higher or lower than the reference models. All models are trained and evaluated on ImageNet-1K.

2.1. Training Techniques

Apart from the design of the network architecture, the training procedure also affects the ultimate performance. Not only did vision Transformers bring a new set of modules and architectural design decisions, but they also introduced different training techniques (*e.g.* AdamW optimizer) to vision. This pertains mostly to the optimization strategy and associated hyper-parameter settings. Thus, the first step of our exploration is to train a baseline model with the vision Transformer training procedure, in this case, ResNet-50/200. Recent studies [7, 81] demonstrate that a set of modern training techniques can significantly enhance the performance of a simple ResNet-50 model. In our study, we use a training recipe that is close to DeiT's [73] and Swin Transformer's [45]. The training is extended to 300 epochs from the original 90 epochs for ResNets. We use the AdamW optimizer [46], data augmentation techniques such as Mixup [90], Cutmix [89], RandAugment [14], Random Erasing [91], and regularization schemes including Stochastic Depth [36] and Label Smoothing [69]. The complete set of hyper-parameters we use can be found in Appendix A.1. By itself, this enhanced training recipe increased the performance of the ResNet-50 model from 76.1% [1] to 78.8% (+2.7%), implying that a significant portion of the performance difference between traditional ConvNets and vision Transformers may be due to the training techniques. We will use this fixed training recipe with the same hyperparameters throughout the "modernization" process. Each reported accuracy on the ResNet-50 regime is an average obtained from training with three different random seeds.

2.2. Macro Design

We now analyze Swin Transformers' macro network design. Swin Transformers follow ConvNets [28, 65] to use a multi-stage design, where each stage has a different feature map resolution. There are two interesting design considerations: the stage compute ratio, and the "stem cell" structure.

Changing stage compute ratio. The original design of the computation distribution across stages in ResNet was largely empirical. The heavy "res4" stage was meant to be compatible with downstream tasks like object detection, where a detector head operates on the 14×14 feature plane. Swin-T, on the other hand, followed the same principle but with a slightly different stage compute ratio of 1:1:3:1. For larger Swin Transformers, the ratio is 1:1:9:1. Following the design, we adjust the number of blocks in each stage from (3, 4, 6, 3) in ResNet-50 to (3, 3, 9, 3), which also aligns the FLOPs with Swin-T. This improves the model accuracy from 78.8% to 79.4%. Notably, researchers have thoroughly investigated the distribution of computation [53, 54], and a more optimal design is likely to exist.

From now on, we will use this stage compute ratio.

Changing stem to "Patchify". Typically, the stem cell design is concerned with how the input images will be processed at the network's beginning. Due to the redundancy

inherent in natural images, a common stem cell will aggressively downsample the input images to an appropriate feature map size in both standard ConvNets and vision Transformers. The stem cell in standard ResNet contains a 7×7 convolution layer with stride 2, followed by a max pool, which results in a 4× downsampling of the input images. In vision Transformers, a more aggressive "patchify" strategy is used as the stem cell, which corresponds to a large kernel size (e.g. kernel size = 14 or 16) and non-overlapping convolution. Swin Transformer uses a similar "patchify" layer, but with a smaller patch size of 4 to accommodate the architecture's multi-stage design. We replace the ResNet-style stem cell with a patchify layer implemented using a 4×4, stride 4 convolutional layer. The accuracy has changed from 79.4% to 79.5%. This suggests that the stem cell in a ResNet may be substituted with a simpler "patchify" layer à la ViT which will result in similar performance.

We will use the "patchify stem" (4×4 non-overlapping convolution) in the network.

2.3. ResNeXt-ify

In this part, we attempt to adopt the idea of ResNeXt [87], which has a better FLOPs/accuracy trade-off than a vanilla ResNet. The core component is grouped convolution, where the convolutional filters are separated into different groups. At a high level, ResNeXt's guiding principle is to "use more groups, expand width". More precisely, ResNeXt employs grouped convolution for the 3×3 conv layer in a bottleneck block. As this significantly reduces the FLOPs, the network width is expanded to compensate for the capacity loss.

In our case we use depthwise convolution, a special case of grouped convolution where the number of groups equals the number of channels. Depthwise conv has been popularized by MobileNet [34] and Xception [11]. We note that depthwise convolution is similar to the weighted sum operation in self-attention, which operates on a per-channel basis, *i.e.*, only mixing information in the spatial dimension. The combination of depthwise conv and 1×1 convs leads to a separation of spatial and channel mixing, a property shared by vision Transformers, where each operation either mixes information across spatial or channel dimension, but not both. The use of depthwise convolution effectively reduces the network FLOPs and, as expected, the accuracy. Following the strategy proposed in ResNeXt, we increase the network width to the same number of channels as Swin-T's (from 64 to 96). This brings the network performance to 80.5% with increased FLOPs (5.3G).

We will now employ the ResNeXt design.

2.4. Inverted Bottleneck

One important design in every Transformer block is that it creates an inverted bottleneck, *i.e.*, the hidden dimension of the MLP block is four times wider than the input dimension

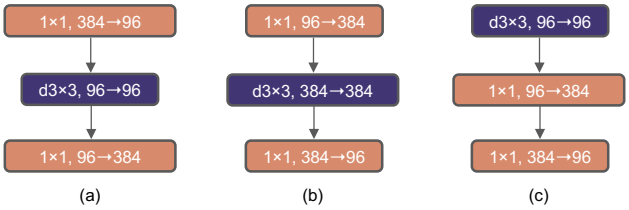


Figure 3. **Block modifications and resulted specifications. (a)** is a ResNeXt block; in **(b)** we create an inverted bottleneck block and in **(c)** the position of the spatial depthwise conv layer is moved up.

(see Figure 4). Interestingly, this Transformer design is connected to the inverted bottleneck design with an expansion ratio of 4 used in ConvNets. The idea was popularized by MobileNetV2 [61], and has subsequently gained traction in several advanced ConvNet architectures [70, 71].

Here we explore the inverted bottleneck design. Figure 3 (a) to (b) illustrate the configurations. Despite the increased FLOPs for the depthwise convolution layer, this change reduces the whole network FLOPs to 4.6G, due to the significant FLOPs reduction in the downsampling residual blocks' shortcut 1×1 conv layer. Interestingly, this results in slightly improved performance (80.5% to 80.6%). In the ResNet-200 / Swin-B regime, this step brings even more gain (81.9% to 82.6%) also with reduced FLOPs.

We will now use inverted bottlenecks.

2.5. Large Kernel Sizes

In this part of the exploration, we focus on the behavior of large convolutional kernels. One of the most distinguishing aspects of vision Transformers is their non-local self-attention, which enables each layer to have a global receptive field. While large kernel sizes have been used in the past with ConvNets [40, 68], the gold standard (popularized by VGGNet [65]) is to stack small kernel-sized (3×3) conv layers, which have efficient hardware implementations on modern GPUs [41]. Although Swin Transformers reintroduced the local window to the self-attention block, the window size is at least 7×7, significantly larger than the ResNe(X)t kernel size of 3×3. Here we revisit the use of large kernel-sized convolutions for ConvNets.

Moving up depthwise conv layer. To explore large kernels, one prerequisite is to move up the position of the depthwise conv layer (Figure 3 (b) to (c)). That is a design decision also evident in Transformers: the MSA block is placed prior to the MLP layers. As we have an inverted bottleneck block, this is a natural design choice — the complex/inefficient modules (MSA, large-kernel conv) will have fewer channels, while the efficient, dense 1×1 layers will do the heavy lifting. This intermediate step reduces the FLOPs to 4.1G, resulting in a temporary performance degradation to 79.9%.

Increasing the kernel size. With all of these preparations, the benefit of adopting larger kernel-sized convolutions is sig-

nificant. We experimented with several kernel sizes, including 3, 5, 7, 9, and 11. The network's performance increases from 79.9% (3×3) to 80.6% (7×7), while the network's FLOPs stay roughly the same. Additionally, we observe that the benefit of larger kernel sizes reaches a saturation point at 7×7. We verified this behavior in the large capacity model too: a ResNet-200 regime model does not exhibit further gain when we increase the kernel size beyond 7×7.

We will use 7×7 depthwise conv in each block.

At this point, we have concluded our examination of network architectures on a macro scale. Intriguingly, a significant portion of the design choices taken in a vision Transformer may be mapped to ConvNet instantiations.

2.6. Micro Design

In this section, we investigate several other architectural differences at a micro scale — most of the explorations here are done at the layer level, focusing on specific choices of activation functions and normalization layers.

Replacing ReLU with GELU One discrepancy between NLP and vision architectures is the specifics of which activation functions to use. Numerous activation functions have been developed over time, but the Rectified Linear Unit (ReLU) [49] is still extensively used in ConvNets due to its simplicity and efficiency. ReLU is also used as an activation function in the original Transformer paper [77]. The Gaussian Error Linear Unit, or GELU [32], which can be thought of as a smoother variant of ReLU, is utilized in the most advanced Transformers, including Google's BERT [18] and OpenAI's GPT-2 [52], and, most recently, ViTs. We find that ReLU can be substituted with GELU in our ConvNet too, although the accuracy stays unchanged (80.6%).

Fewer activation functions. One minor distinction between a Transformer and a ResNet block is that Transformers have fewer activation functions. Consider a Transformer block with key/query/value linear embedding layers, the projection layer, and two linear layers in an MLP block. There is only one activation function present in the MLP block. In comparison, it is common practice to append an activation function to each convolutional layer, including the 1×1 convs. Here we examine how performance changes when we stick to the same strategy. As depicted in Figure 4, we eliminate all GELU layers from the residual block except for one between two 1×1 layers, replicating the style of a Transformer block. This process improves the result by 0.7% to 81.3%, practically matching the performance of Swin-T.

We will now use a single GELU activation in each block.

Fewer normalization layers. Transformer blocks usually have fewer normalization layers as well. Here we remove two BatchNorm (BN) layers, leaving only one BN layer before the conv 1×1 layers. This further *boosts* the performance to 81.4%, already surpassing Swin-T's result. Note

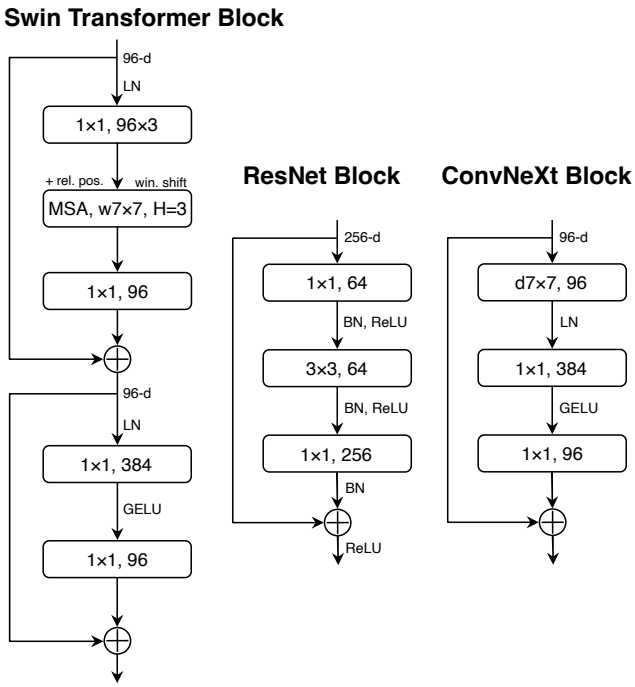


Figure 4. **Block designs** for a ResNet, a Swin Transformer, and a ConvNeXt. Swin Transformer's block is more sophisticated due to the presence of multiple specialized modules and two residual connections. For simplicity, we note the linear layers in Transformer MLP blocks also as "1×1 convs" since they are equivalent.

that we have even fewer normalization layers per block than Transformers, as empirically we find that adding one additional BN layer at the beginning of the block does not improve the performance.

Substituting BN with LN. BatchNorm [38] is an essential component in ConvNets as it improves the convergence and reduces overfitting. However, BN also has many intricacies that can have a detrimental effect on the model's performance [84]. There have been numerous attempts at developing alternative normalization [60, 75, 83] techniques, but BN has remained the preferred option in most vision tasks. On the other hand, the simpler Layer Normalization [5] (LN) has been used in Transformers, resulting in good performance across different application scenarios.

Directly substituting LN for BN in the original ResNet will result in suboptimal performance [83]. With all the modifications in network architecture and training techniques, here we revisit the impact of using LN in place of BN. We observe that our ConvNet model does not have any difficulties training with LN; in fact, the performance is slightly better, obtaining an accuracy of 81.5%.

From now on, we will use one LayerNorm as our choice of normalization in each residual block.

Separate downsampling layers. In ResNet, the spatial downsampling is achieved by the residual block at the start of

each stage, using 3×3 conv with stride 2 (and 1×1 conv with stride 2 at the shortcut connection). In Swin Transformers, a separate downsampling layer is added between stages. We explore a similar strategy in which we use 2×2 conv layers with stride 2 for spatial downsampling. This modification surprisingly leads to diverged training. Further investigation shows that, adding normalization layers wherever spatial resolution is changed can help stablize training. These include several LN layers also used in Swin Transformers: one before each downsampling layer, one after the stem, and one after the final global average pooling. We can improve the accuracy to 82.0%, significantly exceeding Swin-T's 81.3%.

We will use separate downsampling layers. This brings us to our final model, which we have dubbed ConvNeXt.

A comparison of ResNet, Swin, and ConvNeXt block structures can be found in Figure 4. A comparison of ResNet-50, Swin-T and ConvNeXt-T's detailed architecture specifications can be found in Table 9.

Closing remarks. We have finished our first "playthrough" and discovered ConvNeXt, a pure ConvNet, that can outperform the Swin Transformer for ImageNet-1K classification in this compute regime. It is worth noting that *all design choices discussed so far are adapted from vision Transformers.* In addition, *these designs are not novel even in the ConvNet literature — they have all been researched separately, but not collectively, over the last decade.* Our ConvNeXt model has approximately the same FLOPs, #params., throughput, and memory use as the Swin Transformer, but does not require specialized modules such as shifted window attention or relative position biases.

These findings are encouraging but not yet completely convincing — our exploration thus far has been limited to a small scale, but vision Transformers' scaling behavior is what truly distinguishes them. Additionally, the question of whether a ConvNet can compete with Swin Transformers on downstream tasks such as object detection and semantic segmentation is a central concern for computer vision practitioners. In the next section, we will scale up our ConvNeXt models both in terms of data and model size, and evaluate them on a diverse set of visual recognition tasks.

3. Empirical Evaluations on ImageNet

We construct different ConvNeXt variants, ConvNeXt-T/S/B/L, to be of similar complexities to Swin-T/S/B/L [45]. ConvNeXt-T/B is the end product of the "modernizing" procedure on ResNet-50/200 regime, respectively. In addition, we build a larger ConvNeXt-XL to further test the scalability of ConvNeXt. The variants only differ in the number of channels C, and the number of blocks B in each stage. Following both ResNets and Swin Transformers, the number of channels doubles at each new stage. We summarize the configurations below:

- ConvNeXt-T: $C = (96, 192, 384, 768)$, $B = (3, 3, 9, 3)$
- ConvNeXt-S: $C = (96, 192, 384, 768)$, $B = (3, 3, 27, 3)$
- ConvNeXt-B: $C = (128, 256, 512, 1024)$, $B = (3, 3, 27, 3)$
- ConvNeXt-L: $C = (192, 384, 768, 1536)$, $B = (3, 3, 27, 3)$
- ConvNeXt-XL: $C = (256, 512, 1024, 2048)$, $B = (3, 3, 27, 3)$

3.1. Settings

The ImageNet-1K dataset consists of 1000 object classes with 1.2M training images. We report ImageNet-1K top-1 accuracy on the validation set. We also conduct pre-training on ImageNet-22K, a larger dataset of 21841 classes (a superset of the 1000 ImageNet-1K classes) with ∼14M images for pre-training, and then fine-tune the pre-trained model on ImageNet-1K for evaluation. We summarize our training setups below. More details can be found in Appendix A.

Training on ImageNet-1K. We train ConvNeXts for 300 epochs using AdamW [46] with a learning rate of 4e-3. There is a 20-epoch linear warmup and a cosine decaying schedule afterward. We use a batch size of 4096 and a weight decay of 0.05. For data augmentations, we adopt common schemes including Mixup [90], Cutmix [89], RandAugment [14], and Random Erasing [91]. We regularize the networks with Stochastic Depth [37] and Label Smoothing [69]. Layer Scale [74] of initial value 1e-6 is applied. We use Exponential Moving Average (EMA) [51] as we find it alleviates larger models' overfitting.

Pre-training on ImageNet-22K. We pre-train ConvNeXts on ImageNet-22K for 90 epochs with a warmup of 5 epochs. We do not use EMA. Other settings follow ImageNet-1K.

Fine-tuning on ImageNet-1K. We fine-tune ImageNet-22K pre-trained models on ImageNet-1K for 30 epochs. We use AdamW, a learning rate of 5e-5, cosine learning rate schedule, layer-wise learning rate decay [6, 12], no warmup, a batch size of 512, and weight decay of 1e-8. The default pre-training, fine-tuning, and testing resolution is 224^2. Additionally, we fine-tune at a larger resolution of 384^2, for both ImageNet-22K and ImageNet-1K pre-trained models.

Compared with ViTs/Swin Transformers, ConvNeXts are simpler to fine-tune at different resolutions, as the network is fully-convolutional and there is no need to adjust the input patch size or interpolate absolute/relative position biases.

3.2. Results

ImageNet-1K. Table 1 (upper) shows the result comparison with two recent Transformer variants, DeiT [73] and Swin Transformers [45], as well as two ConvNets from architecture search - RegNets [54], EfficientNets [71] and EfficientNetsV2 [72]. ConvNeXt competes favorably with two strong ConvNet baselines (RegNet [54] and EfficientNet [71]) in terms of the accuracy-computation trade-off, as well as the inference throughputs. ConvNeXt also outperforms Swin Transformer of similar complexities *across the*

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
• RegNetY-16G [54]	224^2	84M	16.0G	334.7	82.9
• EffNet-B7 [71]	600^2	66M	37.0G	55.1	84.3
• EffNetV2-L [72]	480^2	120M	53.0G	83.7	85.7
○ DeiT-S [73]	224^2	22M	4.6G	978.5	79.8
○ DeiT-B [73]	224^2	87M	17.6G	302.1	81.8
○ Swin-T	224^2	28M	4.5G	757.9	81.3
• ConvNeXt-T	224^2	29M	4.5G	774.7	**82.1**
○ Swin-S	224^2	50M	8.7G	436.7	83.0
• ConvNeXt-S	224^2	50M	8.7G	447.1	**83.1**
○ Swin-B	224^2	88M	15.4G	286.6	83.5
• ConvNeXt-B	224^2	89M	15.4G	292.1	**83.8**
○ Swin-B	384^2	88M	47.1G	85.1	84.5
• ConvNeXt-B	384^2	89M	45.0G	95.7	**85.1**
• ConvNeXt-L	224^2	198M	34.4G	146.8	**84.3**
• ConvNeXt-L	384^2	198M	101.0G	50.4	**85.5**
ImageNet-22K pre-trained models					
• R-101x3 [39]	384^2	388M	204.6G	-	84.4
• R-152x4 [39]	480^2	937M	840.5G	-	85.4
• EffNetV2-L [72]	480^2	120M	53.0G	83.7	86.8
• EffNetV2-XL [72]	480^2	208M	94.0G	56.5	87.3
○ ViT-B/16 (☎) [67]	384^2	87M	55.5G	93.1	85.4
○ ViT-L/16 (☎) [67]	384^2	305M	191.1G	28.5	86.8
• ConvNeXt-T	224^2	29M	4.5G	774.7	**82.9**
• ConvNeXt-T	384^2	29M	13.1G	282.8	**84.1**
• ConvNeXt-S	224^2	50M	8.7G	447.1	**84.6**
• ConvNeXt-S	384^2	50M	25.5G	163.5	**85.8**
○ Swin-B	224^2	88M	15.4G	286.6	85.2
• ConvNeXt-B	224^2	89M	15.4G	292.1	**85.8**
○ Swin-B	384^2	88M	47.0G	85.1	86.4
• ConvNeXt-B	384^2	89M	45.1G	95.7	**86.8**
○ Swin-L	224^2	197M	34.5G	145.0	86.3
• ConvNeXt-L	224^2	198M	34.4G	146.8	**86.6**
○ Swin-L	384^2	197M	103.9G	46.0	87.3
• ConvNeXt-L	384^2	198M	101.0G	50.4	**87.5**
• ConvNeXt-XL	224^2	350M	60.9G	89.3	**87.0**
• ConvNeXt-XL	384^2	350M	179.0G	30.2	**87.8**

Table 1. **Classification accuracy on ImageNet-1K.** Similar to Transformers, ConvNeXt also shows promising scaling behavior with higher-capacity models and a larger (pre-training) dataset. Inference throughput is measured on a V100 GPU, following [45]. On an A100 GPU, ConvNeXt can have a much higher throughput than Swin Transformer. See Appendix E. (☎)ViT results with 90-epoch AugReg [67] training, provided through personal communication with the authors.

board, sometimes with a substantial margin (*e.g.* 0.8% for ConvNeXt-T). Without specialized modules such as shifted windows or relative position bias, ConvNeXts also enjoy improved throughput compared to Swin Transformers.

A highlight from the results is ConvNeXt-B at 384^2: it outperforms Swin-B by 0.6% (85.1% vs. 84.5%), but with 12.5% higher inference throughput (95.7 vs. 85.1 image/s). We note that the FLOPs/throughput advantage of ConvNeXt-B over Swin-B becomes larger when the resolution increases from 224^2 to 384^2. Additionally, we observe an improved result of 85.5% when further scaling to ConvNeXt-L.

ImageNet-22K. We present results with models fine-tuned from ImageNet-22K pre-training at Table 1 (lower). These experiments are important since a widely held view is that vision Transformers have fewer inductive biases thus can perform better than ConvNets when pre-trained on a larger scale. Our results demonstrate that properly designed ConvNets are *not* inferior to vision Transformers when pre-trained with large dataset — ConvNeXts still perform on par or better than similarly-sized Swin Transformers, with slightly higher throughput. Additionally, our ConvNeXt-XL model achieves an accuracy of 87.8% — a decent improvement over ConvNeXt-L at 384^2, demonstrating that ConvNeXts are scalable architectures.

On ImageNet-1K, EfficientNetV2-L, a searched architecture equipped with advanced modules (such as Squeeze-and-Excitation [35]) and progressive training procedure achieves top performance. However, with ImageNet-22K pre-training, ConvNeXt is able to outperform EfficientNetV2, further demonstrating the importance of large-scale training.

In Appendix B, we discuss robustness and out-of-domain generalization results for ConvNeXt.

3.3. Isotropic ConvNeXt *vs.* ViT

In this ablation, we examine if our ConvNeXt block design is generalizable to ViT-style [20] isotropic architectures which have no downsampling layers and keep the same feature resolutions (*e.g.* 14×14) at all depths. We construct isotropic ConvNeXt-S/B/L using the same feature dimensions as ViT-S/B/L (384/768/1024). Depths are set at 18/18/36 to match the number of parameters and FLOPs. The block structure remains the same (Fig. 4). We use the supervised training results from DeiT [73] for ViT-S/B and MAE [26] for ViT-L, as they employ improved training procedures over the original ViTs [20]. ConvNeXt models are trained with the same settings as before, but with longer warmup epochs. Results for ImageNet-1K at 224^2 resolution are in Table 2. We observe ConvNeXt can perform generally on par with ViT, showing that our ConvNeXt block design is competitive when used in non-hierarchical models.

model	#param.	FLOPs	throughput (image / s)	training mem. (GB)	IN-1K acc.
○ ViT-S	22M	4.6G	978.5	4.9	79.8
• ConvNeXt-S (*iso.*)	22M	4.3G	1038.7	4.2	79.7
○ ViT-B	87M	17.6G	302.1	9.1	81.8
• ConvNeXt-B (*iso.*)	87M	16.9G	320.1	7.7	82.0
○ ViT-L	304M	61.6G	93.1	22.5	82.6
• ConvNeXt-L (*iso.*)	306M	59.7G	94.4	20.4	82.6

Table 2. **Comparing isotropic ConvNeXt and ViT.** Training memory is measured on V100 GPUs with 32 per-GPU batch size.

backbone	FLOPs	FPS	AP^{box}	AP^{box}_{50}	AP^{box}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
Mask-RCNN 3× schedule								
○ Swin-T	267G	23.1	46.0	68.1	50.3	41.6	65.1	44.9
● ConvNeXt-T	262G	25.6	**46.2**	67.9	50.8	**41.7**	65.0	44.9
Cascade Mask-RCNN 3× schedule								
● ResNet-50	739G	16.2	46.3	64.3	50.5	40.1	61.7	43.4
● X101-32	819G	13.8	48.1	66.5	52.4	41.6	63.9	45.2
● X101-64	972G	12.6	48.3	66.4	52.3	41.7	64.0	45.1
○ Swin-T	745G	12.2	50.4	69.2	54.7	43.7	66.6	47.3
● ConvNeXt-T	741G	13.5	**50.4**	69.1	54.8	**43.7**	66.5	47.3
○ Swin-S	838G	11.4	51.9	70.7	56.3	45.0	68.2	48.8
● ConvNeXt-S	827G	12.0	**51.9**	70.8	56.5	**45.0**	68.4	49.1
○ Swin-B	982G	10.7	51.9	70.5	56.4	45.0	68.1	48.9
● ConvNeXt-B	964G	11.4	**52.7**	71.3	57.2	**45.6**	68.9	49.5
○ Swin-B‡	982G	10.7	53.0	71.8	57.5	45.8	69.4	49.7
● ConvNeXt-B‡	964G	11.5	**54.0**	73.1	58.8	**46.9**	70.6	51.3
○ Swin-L‡	1382G	9.2	53.9	72.4	58.8	46.7	70.1	50.8
● ConvNeXt-L‡	1354G	10.0	**54.8**	73.8	59.8	**47.6**	71.3	51.7
● ConvNeXt-XL‡	1898G	8.6	**55.2**	74.2	59.9	**47.7**	71.6	52.2

Table 3. **COCO object detection and segmentation results** using Mask-RCNN and Cascade Mask-RCNN. ‡ indicates that the model is pre-trained on ImageNet-22K. ImageNet-1K pre-trained Swin results are from their Github repository [3]. AP numbers of the ResNet-50 and X101 models are from [45]. We measure FPS on an A100 GPU. FLOPs are calculated with image size (1280, 800).

backbone	input crop.	mIoU	#param.	FLOPs
ImageNet-1K pre-trained				
○ Swin-T	512^2	45.8	60M	945G
● ConvNeXt-T	512^2	**46.7**	60M	939G
○ Swin-S	512^2	49.5	81M	1038G
● ConvNeXt-S	512^2	**49.6**	82M	1027G
○ Swin-B	512^2	49.7	121M	1188G
● ConvNeXt-B	512^2	**49.9**	122M	1170G
ImageNet-22K pre-trained				
○ Swin-B‡	640^2	51.7	121M	1841G
● ConvNeXt-B‡	640^2	**53.1**	122M	1828G
○ Swin-L‡	640^2	53.5	234M	2468G
● ConvNeXt-L‡	640^2	**53.7**	235M	2458G
● ConvNeXt-XL‡	640^2	**54.0**	391M	3335G

Table 4. **ADE20K validation results** using UperNet [85]. ‡ indicates IN-22K pre-training. Swins' results are from its GitHub repository [2]. Following Swin, we report mIoU results with multi-scale testing. FLOPs are based on input sizes of (2048, 512) and (2560, 640) for IN-1K and IN-22K pre-trained models, respectively.

4. Empirical Evaluation on Downstream Tasks

Object detection and segmentation on COCO. We fine-tune Mask R-CNN [27] and Cascade Mask R-CNN [9] on the COCO dataset with ConvNeXt backbones. Following Swin Transformer [45], we use multi-scale training, AdamW optimizer, and a 3× schedule. Further details and hyper-parameter settings can be found in Appendix A.3.

Table 3 shows object detection and instance segmentation results comparing Swin Transformer, ConvNeXt, and traditional ConvNet such as ResNeXt. Across different model complexities, ConvNeXt achieves on-par or better performance than Swin Transformer. When scaled up to bigger models (ConvNeXt-B/L/XL) pre-trained on ImageNet-22K, in many cases *ConvNeXt is significantly better* (*e.g.* +1.0 AP) than Swin Transformers in terms of box and mask AP.

Semantic segmentation on ADE20K. We also evaluate ConvNeXt backbones on the ADE20K semantic segmentation task with UperNet [85]. All model variants are trained for 160K iterations with a batch size of 16. Other experimental settings follow [6] (see Appendix A.3 for more details). In Table 4, we report validation mIoU with multi-scale testing. ConvNeXt models can achieve competitive performance across different model capacities, further validating the effectiveness of our architecture design.

Remarks on model efficiency. Under similar FLOPs, models with depthwise convolutions are known to be slower and consume more memory than ConvNets with only dense convolutions. It is natural to ask whether the design of ConvNeXt will render it practically inefficient. As demonstrated throughout the paper, the inference throughputs of ConvNeXts are comparable to or exceed that of Swin Transformers. This is true for both classification and other tasks requiring higher-resolution inputs (see Table 1,3 for comparisons of throughput/FPS). Furthermore, we notice that training ConvNeXts requires less memory than training Swin Transformers. For example, training Cascade Mask-RCNN using ConvNeXt-B backbone consumes 17.4GB of peak memory with a per-GPU batch size of 2, while the reference number for Swin-B is 18.5GB. In comparison to vanilla ViT, both ConvNeXt and Swin Transformer exhibit a more favorable accuracy-FLOPs trade-off due to the local computations. It is worth noting that this improved efficiency is a result of the *ConvNet inductive bias*, and is not directly related to the self-attention mechanism in vision Transformers.

5. Related Work

Hybrid models. In both the pre- and post-ViT eras, the hybrid model combining convolutions and self-attentions has been actively studied. Prior to ViT, the focus was on augmenting a ConvNet with self-attention/non-local modules [8, 55, 66, 79] to capture long-range dependencies. The original ViT [20] first studied a hybrid configuration, and a large body of follow-up works focused on reintroducing convolutional priors to ViT, either in an explicit [15, 16, 21, 82, 86, 88] or implicit [45] fashion.

Recent convolution-based approaches. Han *et al.* [25] show that local Transformer attention is equivalent to inhomogeneous dynamic depthwise conv. The MSA block in

Swin is then replaced with a dynamic or regular depthwise convolution, achieving comparable performance to Swin. A concurrent work ConvMixer [4] demonstrates that, in small-scale settings, depthwise convolution can be used as a promising mixing strategy. ConvMixer uses a smaller patch size to achieve the best results, making the throughput much lower than other baselines. GFNet [56] adopts Fast Fourier Transform (FFT) for token mixing. FFT is also a form of convolution, but with a global kernel size and circular padding. Unlike many recent Transformer or ConvNet designs, one primary goal of our study is to provide an in-depth look at the process of modernizing a standard ResNet and achieving state-of-the-art performance.

6. Conclusions

In the 2020s, vision Transformers, particularly hierarchical ones such as Swin Transformers, began to overtake ConvNets as the favored choice for generic vision backbones. The widely held belief is that vision Transformers are more accurate, efficient, and scalable than ConvNets. We propose ConvNeXts, a pure ConvNet model that can compete favorably with state-of-the-art hierarchical vision Transformers across multiple computer vision benchmarks, while retaining the simplicity and efficiency of standard ConvNets. In some ways, our observations are surprising while our ConvNeXt model itself is not completely new — many design choices have all been examined separately over the last decade, but not collectively. We hope that the new results reported in this study will challenge several widely held views and prompt people to rethink the importance of convolution in computer vision.

Acknowledgments. We thank Kaiming He, Eric Mintun, Xingyi Zhou, Ross Girshick, and Yann LeCun for valuable discussions and feedback.

References

[1] PyTorch Vision Models. https://pytorch.org/vision/stable/models.html. Accessed: 2021-10-01.

[2] GitHub repository: Swin transformer. https://github.com/microsoft/Swin-Transformer, 2021.

[3] GitHub repository: Swin transformer for object detection. https://github.com/SwinTransformer/Swin-Transformer-Object-Detection, 2021.

[4] Anonymous. Patches are all you need? *Openreview*, 2021.

[5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv:1607.06450*, 2016.

[6] Hangbo Bao, Li Dong, and Furu Wei. BEiT: BERT pre-training of image transformers. *arXiv:2106.08254*, 2021.

[7] Irwan Bello, William Fedus, Xianzhi Du, Ekin Dogus Cubuk, Aravind Srinivas, Tsung-Yi Lin, Jonathon Shlens, and Barret Zoph. Revisiting resnets: Improved training and scaling strategies. *NeurIPS*, 2021.

[8] Irwan Bello, Barret Zoph, Ashish Vaswani, Jonathon Shlens, and Quoc V Le. Attention augmented convolutional networks. In *ICCV*, 2019.

[9] Zhaowei Cai and Nuno Vasconcelos. Cascade R-CNN: Delving into high quality object detection. In *CVPR*, 2018.

[10] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. MMDetection: Open mmlab detection toolbox and benchmark. *arXiv:1906.07155*, 2019.

[11] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *CVPR*, 2017.

[12] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *ICLR*, 2020.

[13] MMSegmentation contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020.

[14] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *CVPR Workshops*, 2020.

[15] Zihang Dai, Hanxiao Liu, Quoc V Le, and Mingxing Tan. Coatnet: Marrying convolution and attention for all data sizes. *NeurIPS*, 2021.

[16] Stéphane d'Ascoli, Hugo Touvron, Matthew Leavitt, Ari Morcos, Giulio Biroli, and Levent Sagun. ConViT: Improving vision transformers with soft convolutional inductive biases. *ICML*, 2021.

[17] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[19] Piotr Dollár, Serge Belongie, and Pietro Perona. The fastest pedestrian detector in the west. In *BMVC*, 2010.

[20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[21] Haoqi Fan, Bo Xiong, Karttikeya Mangalam, Yanghao Li, Zhicheng Yan, Jitendra Malik, and Christoph Feichtenhofer. Multiscale vision transformers. *ICCV*, 2021.

[22] Vitaly Fedyunin. Tutorial: Channel last memory format in PyTorch. https://pytorch.org/tutorials/intermediate/memory_format_tutorial.html, 2021. Accessed: 2021-10-01.

[23] Ross Girshick. Fast R-CNN. In *ICCV*, 2015.

[24] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.

[25] Qi Han, Zejia Fan, Qi Dai, Lei Sun, Ming-Ming Cheng, Jiaying Liu, and Jingdong Wang. Demystifying local vision transformer: Sparse connectivity, weight sharing, and dynamic weight. *arXiv:2106.04263*, 2021.

[26] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. *arXiv:2111.06377*, 2021.

[27] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask R-CNN. In *ICCV*, 2017.

[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *ECCV*, 2016.

[30] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, et al. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *ICCV*, 2021.

[31] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *ICLR*, 2018.

[32] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv:1606.08415*, 2016.

[33] Dan Hendrycks, Kevin Zhao, Steven Basart, Jacob Steinhardt, and Dawn Song. Natural adversarial examples. In *CVPR*, 2021.

[34] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017.

[35] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *CVPR*, 2018.

[36] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[37] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016.

[38] Sergey Ioffe. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models. In *NeurIPS*, 2017.

[39] Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, and Neil Houlsby. Big Transfer (BiT): General visual representation learning. In *ECCV*, 2020.

[40] Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton. Imagenet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[41] Andrew Lavin and Scott Gray. Fast algorithms for convolutional neural networks. In *CVPR*, 2016.

[42] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.

[43] Yann LeCun, Léon Bottou, Yoshua Bengio, Patrick Haffner, et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.

[44] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.

[45] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. 2021.

[46] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[47] Xiaofeng Mao, Gege Qi, Yuefeng Chen, Xiaodan Li, Ranjie Duan, Shaokai Ye, Yuan He, and Hui Xue. Towards robust vision transformer. *arXiv preprint arXiv:2105.07926*, 2021.

[48] Eric Mintun, Alexander Kirillov, and Saining Xie. On interaction between augmentations and corruptions in natural corruption robustness. *NeurIPS*, 2021.

[49] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.

[50] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[51] Boris T Polyak and Anatoli B Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 1992.

[52] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[53] Ilija Radosavovic, Justin Johnson, Saining Xie, Wan-Yen Lo, and Piotr Dollár. On network design spaces for visual recognition. In *ICCV*, 2019.

[54] Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár. Designing network design spaces. In *CVPR*, 2020.

[55] Prajit Ramachandran, Niki Parmar, Ashish Vaswani, Irwan Bello, Anselm Levskaya, and Jonathon Shlens. Stand-alone self-attention in vision models. *NeurIPS*, 2019.

[56] Yongming Rao, Wenliang Zhao, Zheng Zhu, Jiwen Lu, and Jie Zhou. Global filter networks for image classification. *NeurIPS*, 2021.

[57] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.

[58] Henry A Rowley, Shumeet Baluja, and Takeo Kanade. Neural network-based face detection. *TPAMI*, 1998.

[59] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *IJCV*, 2015.

[60] Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *NeurIPS*, 2016.

[61] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*, 2018.

[62] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus, and Yann LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.

[63] Pierre Sermanet, Koray Kavukcuoglu, Soumith Chintala, and Yann LeCun. Pedestrian detection with unsupervised multi-stage feature learning. In *CVPR*, 2013.

[64] Karen Simonyan and Andrew Zisserman. Two-stream convolutional networks for action recognition in videos. In *NeurIPS*, 2014.

[65] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[66] Aravind Srinivas, Tsung-Yi Lin, Niki Parmar, Jonathon Shlens, Pieter Abbeel, and Ashish Vaswani. Bottleneck transformers for visual recognition. In *CVPR*, 2021.

[67] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your vit? data, augmentation, and regularization in vision transformers. *arXiv preprint arXiv:2106.10270*, 2021.

[68] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.

[69] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.

[70] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. Mnasnet: Platform-aware neural architecture search for mobile. In *CVPR*, 2019.

[71] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[72] Mingxing Tan and Quoc Le. Efficientnetv2: Smaller models and faster training. In *ICML*, 2021.

[73] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. *arXiv:2012.12877*, 2020.

[74] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. *ICCV*, 2021.

[75] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv:1607.08022*, 2016.

[76] Régis Vaillant, Christophe Monrocq, and Yann Le Cun. Original approach for the localisation of objects in images. *Vision, Image and Signal Processing*, 1994.

[77] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[78] Haohan Wang, Songwei Ge, Eric P Xing, and Zachary C Lipton. Learning robust global representations by penalizing local predictive power. *NeurIPS*, 2019.

[79] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *CVPR*, 2018.

[80] Ross Wightman. GitHub repository: Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.

[81] Ross Wightman, Hugo Touvron, and Hervé Jégou. Resnet strikes back: An improved training procedure in timm. *arXiv:2110.00476*, 2021.

[82] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. *ICCV*, 2021.

[83] Yuxin Wu and Kaiming He. Group normalization. In *ECCV*, 2018.

[84] Yuxin Wu and Justin Johnson. Rethinking "batch" in batchnorm. *arXiv:2105.07576*, 2021.

[85] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *ECCV*, 2018.

[86] Tete Xiao, Mannat Singh, Eric Mintun, Trevor Darrell, Piotr Dollár, and Ross Girshick. Early convolutions help transformers see better. In *NeurIPS*, 2021.

[87] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017.

[88] Weijian Xu, Yifan Xu, Tyler Chang, and Zhuowen Tu. Co-scale conv-attentional image transformers. *ICCV*, 2021.

[89] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[90] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[91] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI*, 2020.

[92] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ADE20K dataset. *IJCV*, 2019.